CMPE3209 Computer Organization — Dönem Projesi

M1: Adım 0 (Kişiye Özel Parametreler) ve Adım 1 (Görev Seçimi ve C Programı)

Ad Soyad:	Mehmet Batuhan Özer
Öğrenci numarası:	5001230025
Seçilen görev:	G5 — Run-length sıkıştırma
Teslim tarihi:	2 Kasım 2026, Pazartesi

0. Araç ve derleme bilgileri

Alan	Değer
Seçilen ISA	x86-64 (Microsoft x64 ABI, PE/COFF)
Derleyici	clang-cl (LLVM 21.1.8), MSVC 14.44.35207 araç zinciri
Derleme sistemi	CMake 3.31.6 + Ninja 1.12.1, `CMakeLists.txt` + `CMakePresets.json`
Derleme (`-O0` karşılığı)	`cmake --preset od` → `build-od/rle.exe` (`/Zi /Od /Ob0`)
Derleme (`-O1`)	`cmake --preset o1` → `build-o1/rle.exe` (`/Zi /O1`)
Linker bayrakları	`/DYNAMICBASE:NO /INCREMENTAL:NO /MAP /DEBUG`
Disassembler	`llvm-objdump` 21.1.8, `--x86-asm-syntax=intel`
Debugger	Visual Studio 2022 (değer okuma, kaynak seviyesinde stepping)
Instruction izleyici	`cdb.exe` 10.0.26100.6584 (Windows SDK) — Bölüm 4'teki dinamik ölçüm
Cache simülatörü	Aşama 4'te kendi yazacağım simülatör (kaynak kodu teslim edilecek)
Git deposu adresi	`https://github.com/MBatuhanOzer/Computer_Organization_Project`
E-posta	`batuhanozer1042@gmail.com`

Derleme bayrakları hakkında not. MSVC tarzı sürücüde `/Od` optimizasyon kapalı demektir ve yönergedeki `-O0`'ın karşılığıdır; `/O1` doğrudan karşılıktır. Raporun bütün adresleri şu üç linker ayarına bağlıdır:

- `/DYNAMICBASE:NO` — ASLR kapalı. Bu olmadan PE imajı her çalıştırmada farklı bir tabana yüklenir ve `/MAP` dosyasındaki adresler ile debugger'da görülen adresler tutmaz (gcc'deki `-no-pie` karşılığı).
- `/INCREMENTAL:NO` — artımlı linkleme kapalı. Açık olduğunda linker her fonksiyon için bir ILT thunk'ı üretir ve `call rle_encode` önce bir `jmp`'e, oradan fonksiyona gider; Aşama 3'teki instruction cycle izleme tablosuna kaynakta bulunmayan sahte bir `jmp` girerdi. Bu bayrakla `call` doğrudan fonksiyonun kendisine gider.
- `/MAP` — sembol adresleri. PE imajında sembol tablosu bulunmadığı için `llvm-objdump` fonksiyon adlarını tek başına gösteremez; kernel ve diziler bu dosyadan bulunur.

1. Adım 0 — Kişiye özel parametreler

1.1 Taban büyüklükler

Yönergenin 2. bölümü (s. 1): **d** = numaranın son hanesi, **D2** = son iki hane (00–99), **S** = tüm hanelerin toplamı. SID = 5001230025, haneleri 5, 0, 0, 1, 2, 3, 0, 0, 2, 5.

Sembol	Tanım (yönerge s. 1)	Hesap	Sonuç
d	numaranın son hanesi	5001230025	**5**
D2	son iki hane (00–99)	5001230025	**25**
S	hanelerin toplamı	5+0+0+1+2+3+0+0+2+5	**18**

1.2 Türetilen parametreler

Formüllerin tamamı yönergenin s. 2'deki parametre tablosundan alınmıştır. "(varsayım)" etiketi yönergenin kendi ifadesidir: *"f, T1, T2 ve Nc varsayımsal model parametreleridir, ölçüm değildir."* (s. 2)

Parametre	Formül (yönergeden)	Yerine koyma	Sonuç
Dizi boyu N	$N = 24 + (D2 \bmod 25)$	$24 + (25 \bmod 25) = 24 + 0$	**24 eleman**
Cache konfig.	d'ye göre tablodan (s. 2)	d = 5 satırı	**256 B / 16 B line / 4-way / 16 line / 4 set**
Write policy	d çift → write-back; d tek → write-through	d = 5, tek	**write-through**
Interrupt anı k	$k = (S \bmod 7) + 3$	$(18 \bmod 7) + 3 = 4 + 3$	**k = 7**
Interrupt türü	S çift → Timer; S tek → I/O	S = 18, çift	**Timer**
Saat frekansı f	$f = 1.0 + 0.1 \times d$ GHz	$1.0 + 0.1 \times 5$	**1.5 GHz** (varsayım)
Erişim süresi T2	$T2 = 20 + 5 \times (d \bmod 4)$ cycle	$20 + 5 \times 1$	**25 cycle** (varsayım)
Erişim süresi T1	$T1 = 1$ cycle	—	**1 cycle** (varsayım)
Çekirdek sayısı Nc	$Nc = 2 + (S \bmod 7)$	$2 + 4$	**6** (varsayım)
Görev param. R	G5: $R = 2 + (SID \bmod 3)$ (s. 4)	hane toplamı 18, $3 \mid 18 \Rightarrow SID \bmod 3 = 0$	**R = 2**
Görev param. B	G5: $B = 0$ (s. 4)	—	**B = 0**

Türetilmiş: $\tau = 1/f = 1/(1{,}5\text{ GHz}) = \mathbf{0{,}667}$ ns (Aşama 5a'da kullanılacak).

Cache satırının tutarlılık kontrolü. $256\text{ B}/16\text{ B} = 16$ line; $16\text{ line}/4\text{ yol} = 4$ set — yönergenin d = 5 satırındaki "line sayısı 16, set sayısı 4" değerleriyle örtüşüyor. ✓

1.3 Adres alanlarının türetilmesi

Yönerge s. 8 (4b): *"set-associative'de tag = s − d, set = d, word = w bit."* Tutanak: *"ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır."* x86-64 için $s + w = 32$.

Alan	Türetme	Bit	Konum
Word (w)	line boyu $= 16\text{ B} = 2^4 \Rightarrow w = 4$	**4**	adres[3:0]
s	$s + w = 32 \Rightarrow s = 32 - 4$	28	—
Set (d)	set sayısı $= 4 = 2^2 \Rightarrow d = 2$	**2**	adres[5:4]
Tag (s − d)	$28 - 2$	**26**	adres[31:6]
Toplam	$26 + 2 + 4$	**32**	—

4-way set-associative olduğu için *line* değil *set* alanı kullanılır. Direct mapping olsaydı $r = \log_2 16 = 4$ ve tag $= 28 - 4 = 24$ bit olurdu; bu değer Aşama 4e'nin mapping taramasında gerekecektir.

1.4 SID'in programdaki biçimi

Yönerge s. 3: *"Numara $2^{31}-1$'i aşıyorsa son 9 hanesi kullanılır."* $5\,001\,230\,025 > 2\,147\,483\,647$ olduğundan son 9 hane: $\mathbf{SID_{program}} = \mathbf{1\,230\,025}$ = `0x12C4C9`.

Doğrulama: 0x12C4C9 $= 1048576 + 131072 + 49152 + 1024 + 192 + 9 = 1\,230\,025$ ✓

Adım 0'daki d, D2 ve S **tam numaradan** hesaplanmıştır (bkz. Bölüm 5, Soru 2).

2. Adım 1 — Görev seçimi ve C programı

2.1 Görev seçimi

☒ **G5 — Run-length sıkıştırma.** $R = 2$, $B = 0 \Rightarrow$ sembol alfabesi $\{0, 1\}$; dizi boyu $N = 24$.

2.2 Çözüm tasarımı

Program üç global `s32` dizisi üzerinde çalışır: `v[24]` LCG ile üretilen $\{0, 1\}$ sembollerini, `enc[24]` kodlanmış (değer, tekrar) çiftlerini, `dec[24]` ise geri açılmış diziyi tutar. `rle_encode(n)` `v` dizisini tek geçişte tarar; dış döngü her koşu için bir kez döner, iç döngü aynı sembolün tekrar sayısını sayar ve her koşu `(v[i] << 8) | run` biçiminde tek bir 32-bit word'e paketlenerek `enc` dizisine yazılır — böylece shift, OR ve dizi store'u aynı satırda görünür hale gelir. `rle_decode(len)` bu paketleri `>> 8` ve `& 0xFF` ile ayrıştırıp `dec` dizisine açar; her iki fonksiyon da parametre alıp değer döndürür (Y3). Doğrulama iki aşamalıdır: önce eleman *sayısı* (`out == N`), sonra `main` içindeki üçüncü geçişte eleman *değerleri* karşılaştırılır. Veri yerleşimi bilinçli olarak üç ayrı global dizi şeklinde seçilmiştir: üç dizi 15 cache line'a karşılık gelir, 256 B'lık cache ise 16 line tutar — bu, 4-way set'lerden tam birinin taşmasını sağlayarak Aşama 4'te hem spatial locality'nin hem de conflict miss'in aynı iz üzerinde gözlenmesine imkân verir (bkz. Bölüm 4.2).

2.3 C kaynak kodu

Dosya: `main.c` — tek dosya, C99, **55 satır kod** (toplam 70 satır; 9 boş satır ve 6 yorum satırı hariç). Yönergenin 25–60 satır aralığı içindedir. Satır sayımı bu belgede "boş ve yalnızca yorum içeren satırlar hariç" olarak tanımlanmıştır.

Program `s32` ve `u32` typedef'lerini kullanır; bunlar sırasıyla `int` ve `unsigned int` takma adlarıdır, dolayısıyla yönergenin "32 bitten geniş tamsayı türü" yasağına girmez ve üretilen makine kodunu değiştirmez. `Y6`'nın istediği `const int SID` bildirimi programda `const s32 SID` olarak yer alır (`s32` ≡ `int`).

Yasak yapı taraması (yönerge s. 5):

Aranan	Sonuç
`printf scanf puts` (kütüphane çağrısı)	bulunamadı
`float double` (kayan nokta)	bulunamadı
`malloc calloc free` (dinamik bellek)	bulunamadı
`long long int64 uint64` (>32 bit)	bulunamadı
`#include`	yok — hiçbir başlık dosyası kullanılmaz

Derleme uyarısızdır: `clang -std=c99 -pedantic-errors -Wall -Wextra` ve `gcc -std=c99 -pedantic-errors -Wall -Wextra`. Yorumlar C99'un `//` satır yorumu biçimindedir (C99 §6.4.9).

2.4 Zorunlu yapı eşlemesi (Y1–Y7)

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	**15**; 56, 28, 64	`s32 v[N];` — `v` üzerinde üç geçiş: s. 56 yazma (1.), s. 28 okuma/kodlama (2.), s. 64 doğrulama (3.)
Y2	En az bir koşullu dallanma (if)	**30**; 65	s. 30: `v[i+run] == v[i]` koşu uzunluğunu belirler; s. 65: `if (dec[i] != v[i])`
Y3	Parametre alan ve değer döndüren fonksiyon çağrısı	**24**, 39 (tanım); **60**, 61 (çağrı)	`s32 rle_encode(s32 n)` ve `s32 rle_decode(s32 len)`; her ikisi de parametre alır ve değer döndürür, dönen değer çağrı yerinde bir değişkene atanır
Y4	Toplama/çıkarma, çarpma veya shift, ve bit işlemi	**31**, 33, 43, 44, 62	s. 31: `<< 8` (shift) ve `| run` (OR); s. 33: `i += run` (toplama); s. 43: `>> 8` (shift); s. 44: `& 0xFF` (AND); s. 62: `100 * enc_len` (çarpma)
Y5	Dizi elemanına en az bir yazma (store)	**58**, 31, 46	`v[i] = ...`, `enc[k] = ...`, `dec[o++] = ...`
Y6	`const int SID` ve sonucu etkileyen hesapta kullanımı	**13** (tanım), **55** (kullanım)	`const s32 SID = 1230025;` LCG'nin x_0 tohumudur $\Rightarrow$ tüm `v` dizisini, dolayısıyla üç Y7 çıktısının tamamını belirler
Y7	Sonuç global değişkende tutulur	**19, 20, 21**	`enc_len`, `ratio_x100`, `decode_ok`

2.5 Program çıktıları

Adresler `build-od/rle.map` dosyasından alınmıştır. Değerler program bittikten sonra Visual Studio 2022'nin Watch penceresinden okunmuştur; ekran görüntüsü depoda `ss_globals.png` dosyasındadır. Watch penceresi onlu taban kullanır (`257` = `0x0101`, `259` = `0x0103`) ve `enc`'in tanımlı 24 elemanının tamamını gösterir; yalnızca ilk `enc_len` = 11 tanesi anlamlıdır.

Global değişken	Adres (hex)	Değer	Ekran görüntüsü
`enc_len`	`0x140075C60`	**11** (`0x0000000B`)	`ss_globals.png`
`ratio_x100`	`0x140075C64`	**45** (`0x0000002D`)	`ss_globals.png`
`decode_ok`	`0x140075C68`	**1** (`0x00000001`)	`ss_globals.png`
`v[24]`	`0x140075C70`	`0001 0001 0000 0111 0010 1110`	`ss_globals.png`
`enc[0..10]`	`0x140075CD0`	`0003 0101 0003 0101 0005 0103 0002 0101 0001 0103 0001`	`ss_globals.png`
`dec[24]`	`0x140075D30`	`v` ile birebir aynı	`ss_globals.png`
`SID`	`0x140062000`	`0x12C4C9` = 1230025	—

Koşuların elle doğrulanması. `v` koşulara ayrıldığında uzunluklar $3, 1, 3, 1, 5, 3, 2, 1, 1, 3, 1$; toplam $= 24$ ✓, koşu sayısı $= 11 =$ `enc_len` ✓. İlk çift: $(0 \ll 8) \,|\, 3 =$ 0x0003 ✓.

`ratio_x100` beklentiyle karşılaştırma (yönerge s. 4: *"ratio_x100 R = 2 için yaklaşık 50 ... Öğrenci kendi sonucunu bu beklentiyle karşılaştırır ve farkı açıklar"*): $R = 2$ için beklenen koşu uzunluğu $R/(R-1) = 2$, beklenen koşu sayısı $24/2 = 12$ ve `ratio_x100` ≈ 50. Ölçülen: 11 koşu, $\lfloor 100 \times 11/24 \rfloor = 45$. Fark,

$N = 24$'ün küçüklüğünden kaynaklanan örneklem dalgalanmasıdır; koşu uzunlukları geometrik dağılımlıdır ve 24 elemanlık tek bir örnekte beklenen değerden ± 1–2 koşu sapma olağandır.

LCG hakkında not. Yönergedeki $>> 16$ kaydırması kritik öneme sahiptir: $m = 2^{32}$ modüllü bir LCG'de i'nci bitin periyodu en fazla 2^{i+1}'dir, yani en düşük bit 0,1,0,1 biçiminde değişir. $R = 2$ olduğu için `% R` tek bir bit seçer; kaydırma olmasaydı `v` dizisi `010101...` olur, her koşu 1 uzunluğunda olurdu ve `ratio_x100` 100 çıkardı. Kaydırma, zayıf alt 16 biti atıp iyi karışmış üst yarıdan örnekler.

2.6 İzlenecek kernel

Programdaki iki fonksiyonun (`rle_encode`, `rle_decode`) gövdesi 50 instruction'ı aştığı için, çağrıyı içeren bir blok 12–20 instruction'lık pencereye sığmaz. Bu nedenle kernel bir fonksiyon **dönüşü** etrafında seçilmiştir; yönerge s. 5 buna izin verir: "… fonksiyon çağrısını **ya da** dönüşünü içermelidir". Seçilen blok `rle_decode`'un epilogue'u ve `ret`'i ile `main`'deki izleyen deyimlerdir.

Alan	Değer
C satır aralığı	s. 49 (`return o;`) ve s. 61–64
Başlangıç / bitiş adresi	`0x140001168` – `0x140001232`
Dinamik instruction sayısı	**17** (yönerge aralığı: 12–20)
Toplam boyut	68 byte (instruction başına 1–8 byte)
Hangi iterasyon	`rle_decode`'dan tek dönüş; doğrulama döngüsünün ilk testi ($i = 0$)

#	Adres	Instruction	Not
1	`0x140001168`	`mov eax, dword ptr [rsp+0x10]`	**load** (dönüş değeri `o`)
2	`0x14000116C`	`add rsp, 0x18`	epilogue, frame yıkımı
3	`0x140001170`	`ret`	**fonksiyon dönüşü**
4	`0x1400011F8`	`mov dword ptr [rsp+0x28], eax`	**store** (`out`)
5	`0x1400011FC`	`imul eax, dword ptr [rip+...], 0x64`	**ALU** (çarpma) + load (`enc_len`)
6	`0x140001203`	`mov ecx, 0x18`	bölen = N
7	`0x140001208`	`cdq`	**interrupt burada gelir (k = 7)**
8	`0x140001209`	`idiv ecx`	**ALU** (bölme)
9	`0x14000120B`	`mov dword ptr [rip+...], eax`	**store** ($\rightarrow$ `ratio_x100`)
10	`0x140001211`	`cmp dword ptr [rsp+0x28], 0x18`	load + karşılaştırma
11	`0x140001216`	`sete al`	bayrak maddeleştirme (branch-less)
12	`0x140001219`	`and al, 0x1`	**ALU** (bit işlemi)
13	`0x14000121B`	`movzx eax, al`	sıfır genişletme
14	`0x14000121E`	`mov dword ptr [rip+...], eax`	**store** ($\rightarrow$ `decode_ok`)
15	`0x140001224`	`mov dword ptr [rsp+0x24], 0x0`	döngü sayacı ilklendirme
16	`0x14000122C`	`cmp dword ptr [rsp+0x24], 0x18`	load + karşılaştırma
17	`0x140001231`	`jge 0x14000126C`	**koşullu branch** (alınmaz: $0 < 24$)

Yönergenin kernel şartlarının kontrolü (s. 5): load ✓(#1, #5, #10, #16) — store ✓(#4, #9, #14) — ALU ✓(#5 `imul`, #8 `idiv`, #12 `and`) — koşullu branch ✓(#17) — fonksiyon dönüşü ✓(#3).

$k = 7$ **senaryosu (Aşama 3c girdisi).** Interrupt, kernel'in 7. instruction'ı olan `cdq` (`0x140001208`) execute edilirken gelir; kaydedilecek PC = 8. instruction'ın adresi = `0x140001209`. `cdq`, `EAX`'i `EDX:EAX` çiftine işaret genişletir ve bir sonraki `idiv`'in bölüneni olarak hazırlar. Interrupt tam olarak işaret genişletme ile bölme *arasında* geldiği için kaydedilen bağlamın hem `EDX`'i hem `EAX`'i içermesi zorunludur; aksi halde bölünen bozulur ve `ratio_x100` yanlış hesaplanır. Bu, Aşama 3c'deki "kaydedilen diğer durum bilgisi" sorusuna somut bir cevap oluşturur.

3. Seçilen ISA ve model varsayımları

Yönergenin 4. bölümü (s. 5) ISA seçimini öğrenciye bırakır; x86-64 için elle kodlanacak minimum 8 instruction'dır (sabit uzunluklu ISA'larda 10). Seçim: **x86-64**.

Gerekçe: x86-64 assembly üzerinde çalışma deneyimim var ve Aşama 3, 4, 5'in tamamı okunan assembly üzerinde yürür. Ayrıca kernel'in 5. instruction'ı (`imul eax, dword ptr [rip+...], 0x64` = `6b 05 7d 6d 09 00 64`) tek başına opcode, ModR/M, RIP-relative displacement ve 8-bit immediate alanlarını içerir; girdi üretimindeki `imul eax, dword ptr [rsp+0x30], 0x19660d` ise opcode, ModR/M, SIB, disp8 ve imm32 alanlarının beşini birden gösterir — yönergenin 2c maddesindeki "prefix, opcode, ModR/M, SIB, displacement ve immediate alanları ayrı ayrı gösterilir" şartı bu örneklerle karşılanacaktır. Son olarak bellek operandlı ALU instruction'ları (ör. `add dword ptr [rsp+0x24], 0x1`) tek bir instruction olarak load + ALU + store yapar, yani üç bus işlemi üretir; tutanak 3a bunu açıkça destekler ve bu, Aşama 5b'deki architecture/organization tablosu için sabit uzunluklu bir RISC ISA'da bulunmayan malzeme sağlar.

3.1 Modelin gerçek donanımdan ayrıldığı yerler

Yönergenin Aşama 3 modeli bilinçli bir basitleştirmedir. Raporda şu noktalar belirtilecektir:

- **PC, IR, MAR ve MBR yalnızca kavramsaldır** (yönerge s. 6). x86-64'te MAR ve MBR adlı mimari register yoktur; bunlar debugger'da okunamaz, instruction semantiğinden türetilir. 3a tablosunun yalnızca PC ve değişen register sütunları gözlemdir, kalan sütunlar çıkarımdır.
- **Pipeline yok, her an tek instruction.** Gerçek bir x86-64 çekirdeği out-of-order ve superscalar çalışır; bu kernel'deki 17 instruction gerçek donanımda birkaç cycle'da biter.
- **Mikro-op ayrıştırması ve füzyon yok.** Bellek operandlı `add` gerçek donanımda load + add + store mikro-op'larına ayrılır, `cmp`+`jge` çifti ise çoğu zaman makro-füzyonlanır; modelde her instruction bölünmeden, seri olarak ele alınır.
- **Gerçek line boyu 64 B, model 16 B.** Projede zorunlu tutulan 16 B gerçek L1D'nin dörtte biridir. Aşama 4e'nin line-boyu taraması 8/16/32/64 B'yi kapsadığı için gerçek değer zaten ölçülecek ve 16 B ile karşılaştırılacaktır.
- **Donanım prefetcher ve çok seviyeli hiyerarşi yok**; modelde tek seviyeli bir data cache vardır, instruction fetch'ler cache dışıdır (yönerge s. 2).
- **Data bus genişliği 64 bit** (x86-64 word'ü); adres alanı ayrıştırması ise adresin düşük 32 biti üzerinden yapılır.

4. Aşama 4'ün `/O1` izi üzerinden yapılması talebi

Yönergenin "Öğretim üyesi notları" bölümü 6. madde (s. 12): *"`-O0` izi stack'e boğulur. `-O0` her değişkeni stack'e yazar; cache sonuçları dizi erişimlerinden çok stack trafiğini yansıtabilir. Alternatif: Aşama 4'te `-O1` izini zorunlu tutmak. Pedagojik seçimdir; ilandan önce karar verilmeli."*

Bu programda fark ölçülmüştür. **Ölçüm yöntemi:** `cdb` ile `main`'e breakpoint konup program tek tek (`t`) yürütülmüş, her adımda `rip` kaydedilmiş; kayıtlar yalnızca kendi üç fonksiyonumuzun adres aralığına filtrelenip `main`'in `ret`'inde kesilmiş; ardından her adres `llvm-objdump` çıktısıyla eşleştirilip bellek operandı bölgesine göre sınıflandırılmıştır (stack = `[rsp/rbp±N]`; dizi = `v/enc/dec`; global = `enc_len/ratio_x100/decode_ok`).

Ölçü (dinamik, kendi kodumuz)	**`/Od`**	**`/O1`**
Yürütülen instruction sayısı	2088	988
Toplam data bellek referansı	1152	173
stack referansı	**961 (%83,4)**	**17 (%9,8)**
dizi referansı (`v/enc/dec`)	186 (%16,1)	**152 (%87,9)**
global referansı	5 (%0,4)	4 (%2,3)

`/Od`'de bellek referanslarının **%83'ü stack frame'e** aittir; cache analizi büyük ölçüde dizilerin locality'sini değil, yerel değişkenlerin spill trafiğini ölçer. `/O1`'de kalan 17 stack referansı yalnızca kontrol akışına aittir (`call`/`ret` dönüş adresleri ve callee-saved register'lar) ve referansların %88'i gerçek dizi erişimidir.

Tutarlılık kontrolü. Her iki derlemede de dizi *yazma* sayısı **59**'dur: 24 (`v`) + 11 (`enc`) + 24 (`dec`) = 59 — elle türetilebilen bu değerin iki bağımsız optimizasyon seviyesinde aynı çıkması ölçümün doğruluğunu destekler.

Talep: Aşama 4'ün (4a–4g) `/O1` izi üzerinden yapılması. Aşama 2 ve Aşama 3 yönergedeki gibi `/Od` üzerinden yürütülecek, Aşama 2b karşılaştırması her iki sürümü de kapsamaya devam edecektir.

4.1 Veri yerleşimi

`/DYNAMICBASE:NO` ile sabitlenmiş yerleşim (`/Od`). Diziler bildirim sırasına göre bitişiktir:

Dizi	**Adres aralığı**	**Boyut**	**Kapladığı line (16 B)**
`v[24]`	`0x140075C70`–`0x140075CCF`	96 B	6
`enc[0..10]`	`0x140075CD0`–`0x140075CFB`	44 B	3
`dec[24]`	`0x140075D30`–`0x140075D8F`	96 B	6

4.2 Cache set haritası

Set = adres[5:4]. 64-bit adreslerin düşük 32 biti kullanılır.

Set	**Bu sete düşen line'lar**	**Sayı**	**4-way'e sığar mı?**
0	`v@5C80`, `v@5CC0`, `dec@5D40`, `dec@5D80`	4	tam dolar
1	`v@5C90`, `enc@5CD0`, `dec@5D50`	3	evet
2	`v@5CA0`, `enc@5CE0`, `dec@5D60`	3	evet
3	`v@5C70`, `v@5CB0`, `enc@5CF0`, `dec@5D30`, `dec@5D70`	**5**	**HAYIR — LRU eviction**

Toplam 15 line, kapasite 16 line. Bu $5/4/3/3$ dağılımı dört farklı yerleşimde (gcc `-O0`, gcc `-O1`, clang-cl artımlı linkli, clang-cl `/INCREMENTAL:NO`) aynı çıkmıştır; yalnızca *hangi* setin taştığı değişir. Dolayısıyla bu, derleyiciye değil, 4 setlik bir cache karşısında üç adet 96 byte'lık dizinin yerleşimine ait bir özelliktir. Conflict miss'lerin tek bir sette yoğunlaşması Aşama 4c ve 4e için temiz ve açıklanabilir bir tablo verir.

Write policy'nin sonucu. Write-through olduğu için 4c tablosundaki "Dirty (write-back)" sütunu bu projede boştur (write-through'da dirty biti yoktur) ve ana belleğe yazma sayısı doğrudan store sayısına eşittir (yönerge s. 8). Write miss'te write-allocate varsayılır (yönerge s. 2).

5. Öğretim üyesine sorular

Soru 1 — Aşama 4 izi `/O1`'den alınabilir mi? Bölüm 4'teki ölçümlere dayanarak talep edilmektedir.

Soru 2 — d, D2 ve S hangi numaradan hesaplanacak? Numaram $2^{31} - 1$'i aştığı için yönerge programda son 9 hanenin kullanılmasını söylüyor (`SID = 1230025`). Ancak d, D2 ve S'nin tam numaradan mı kısaltılmış numaradan mı hesaplanacağı belirtilmemiş:

	Tam numara (5001230025)	**Son 9 hane (1230025)**
d	5	5
D2	25	25
S	**18**	13
k	**7**	6
Interrupt türü	**Timer** (S çift)	I/O (S tek)
Nc	**6**	8

Bu teslimde **tam numara** esas alınmıştır (S = 18, k = 7, Timer, Nc = 6); kısaltma yalnızca programdaki `const s32 SID` sabitine uygulanmıştır. Yönerge d ve D2'yi "numaranın" haneleri olarak tanımladığı ve kısaltmayı yalnızca program sabiti bağlamında andığı için bu okuma tercih edilmiştir. Onaylanması rica olunur: yanlış parametre, ona dayanan tüm adımları geçersiz kılmaktadır.

Soru 3 — Değişken uzunluklu instruction'larda instruction fetch kaç bus işlemi sayılır? Yönerge, instruction fetch'leri cache dışında tutuyor (s. 2) ama bus işlemi olarak saymamızı istiyor (s. 7, 3d.1). Aşama 3 modelinde data bus genişliği x86-64 için 64 bittir. Kernel'imin 17 instruction'ı toplam 68 byte tutuyor ve instruction boyları 1 ile 8 byte arasında değişiyor (`ret` 1 byte, `mov dword ptr [rsp+0x24], 0x0` 8 byte). "Instruction başına bir fetch" sayılırsa **17**, "64-bit bus başına 8 byte" sayılırsa yaklaşık **9** bus işlemi çıkıyor; ayrıca instruction'lar 8 byte sınırlarını aştığı için ikinci okuma temiz de değil. Sabit 32 bitlik bir ISA'da bu belirsizlik doğmuyor. Aksi belirtilmedikçe **instruction başına bir fetch bus işlemi** (Fig. 3.2 kavramsal modeli) varsayacağım ve gerçek bir x86 front end'inin 16–32 byte'lık bloklar getirdiğini model-fark paragrafında belirteceğim — 3a ve 3d'yi bu varsayım üzerine kurmadan önce onay rica ediyorum.

Ek — Doğrulama komutları

```
:: x64 Developer Command Prompt icinde (ya da dogrudan Visual Studio: Open Folder)
cmake --preset od   &&  cmake --build --preset od     :: -> build-od/rle.exe
cmake --preset o1   &&  cmake --build --preset o1     :: -> build-o1/rle.exe

llvm-objdump -d --x86-asm-syntax=intel build-od\rle.exe > asm_O0.s
llvm-objdump -d --x86-asm-syntax=intel build-o1\rle.exe > asm_O1.s
findstr /C:"rle_encode" /C:"rle_decode" /C:"main" build-od\rle.map

:: global degerleri bellekten okumak (L sayilari hex: L18 = 24, Lb = 11)
cdb -c "bu rle!main; g; pt; dd rle!enc_len L3; dd rle!v L18; dd rle!enc Lb; dd rle!dec L18; q" ^
    build-od\rle.exe

:: standart C99 uyumlulugu
```

```
clang -std=c99 -pedantic-errors -Wall -Wextra -c main.c
```